



**MATEMATICKO-FYZIKÁLNÍ
FAKULTA**
Univerzita Karlova

BAKALÁŘSKÁ PRÁCE

Stanislav Dochynets

Rozhraní pro plantri a geng

Informatický ústav Univerzity Karlovy

Vedoucí bakalářské práce: prof. Mgr. Zdeněk Dvořák, Ph.D.

Studijní program: Informatika

Studijní obor: Programování a vývoj software

Praha 2026

Prohlašuji, že jsem tuto bakalářskou práci vypracoval(a) samostatně a výhradně s použitím citovaných pramenů, literatury a dalších odborných zdrojů. Beru na vědomí, že se na moji práci vztahují práva a povinnosti vyplývající ze zákona č. 121/2000 Sb., autorského zákona v platném znění, zejména skutečnost, že Univerzita Karlova má právo na uzavření licenční smlouvy o užití této práce jako školního díla podle § 60 odst. 1 autorského zákona.

V dne

Podpis autora

Dedication. It is nice to say thanks to supervisors, friends, family, book authors and food providers.

Název práce: Rozhraní pro plantri a geng

Autor: Stanislav Dochynets

Ústav: Informatický ústav Univerzity Karlovy

Vedoucí bakalářské práce: prof. Mgr. Zdeněk Dvořák, Ph.D., Informatický ústav Univerzity Karlovy

Abstrakt: Tato práce představuje rozhraní v jazyce C++ pro dva zavedené generátory grafů, geng a plantri. Tyto nástroje umožňují efektivní generování obecných a rovinných grafů, jejich nativní rozhraní v jazyce C se však obtížně propojuje s moderním C++ kódem a grafovými algoritmy. Práce navrhuje a implementuje sjednocený wrapper, který zpřístupňuje oba generátory pomocí společného typově bezpečného API. Vygenerované grafy jsou reprezentovány lehkými views na interní data generátorů, což zabraňuje kopírování a umožňuje filtrování, prořezávání a zpracování výstupu během generování. Rozhraní je kompatibilní s knihovnou Boost Graph Library, takže uživatelé mohou na vygenerované grafy přímo použít běžné grafové algoritmy. Knihovna usnadňuje použití výkonného generování grafů v experimentálním softwaru a v algoritmické teorii grafů.

Klíčová slova: generování grafů, obecné grafy, rovinné grafy, barvení grafů, geng, plantri

Title: Interface for plantri and geng

Author: Stanislav Dochynets

Institute: Computer Science Institute of Charles University

Supervisor: prof. Mgr. Zdeněk Dvořák, Ph.D., Computer Science Institute of Charles University

Abstract: This thesis presents a C++ interface for two established graph generators, nauty/geng and plantri. These tools provide efficient generation of general graphs and planar graphs, but their native C interfaces are difficult to combine with modern C++ code and graph algorithms. The thesis designs and implements a unified wrapper that exposes both generators through a common, type-safe API. Generated graphs are represented by lightweight views of the internal generator data, which avoids copying and allows callback-based filtering, pruning, and output processing during generation. The interface is compatible with the Boost Graph Library, so users can apply standard graph algorithms directly to generated graphs. The resulting library makes high-performance graph generation easier to use in experimental and algorithmic graph theory software.

Keywords: graph generation, general graphs, planar graphs, graph coloring, geng, plantri

Obsah

Úvod	3
1 Teoretický úvod	5
1.1 Počítačem podporovaná teorie grafů	5
1.2 Generování grafů	5
1.3 Izomorfismus grafů	6
1.4 Kanonické reprezentace	6
1.5 Backtracking a prořezávání	7
1.6 Praktické komplikace generování grafů	7
2 Generátory grafů geng a plantri	9
2.1 Generátor geng	9
2.2 Generátor plantri	10
2.3 Možnosti rozšíření pomocí callbacků	11
2.4 Omezení původních rozhraní	11
3 Návrh rozhraní	13
3.1 Cíle návrhu	13
3.2 Jednotné rozhraní pro více generátorů	13
3.3 Callbacky pro filtrování a zpracování grafů	14
3.4 View na graf bez kopírování dat	14
3.5 Integrace s Boost Graph Library	14
3.6 Podpora bohatších struktur	15
4 Implementace	17
4.1 Architektura knihovny	17
4.2 Vrstva Generator a backendy	18
4.3 Propojení s geng	18
4.4 Propojení s plantri	19
4.5 Implementace GraphView	19
4.5.1 Integrace s Boost Graph Library	19

4.6	UML diagram tříd	20
4.7	Obarvené a zakotvené vrcholy v geng	20
4.8	Callbacks a přechod mezi C a C++	20
4.9	Výstupní formáty	20
5	Testování a vyhodnocení	21
	Conclusion	23
	Seznam použité literatury	25
A	Using CoolThesisSoftware	27

Úvod

Generování grafů je důležitým nástrojem v experimentální teorii grafů a při testování grafových algoritmů. Umožňuje vytvářet množiny grafů splňujících zadané podmínky a následně na nich ověřovat hypotézy, hledat protipříklady nebo porovnávat chování algoritmů.

Pro generování grafů existují specializované nástroje, například geng z balíku nauty a plantri. Tyto programy jsou velmi efektivní a používají se pro generování různých tříd grafů. Tyto nástroje lze snadno spustit z příkazové řádky, ale jejich rozšiřování vyžaduje psaní nízkoúrovňových pluginů a znalost interních datových struktur jednotlivých generátorů.

Cílem této práce je navrhnout a implementovat jednotné C++ rozhraní pro generátory geng a plantri. Rozhraní má umožnit pohodlnou konfiguraci generování, vlastní filtrování grafů, zpracování výstupu a použití vybraných algoritmů z knihovny Boost Graph Library. Důraz je kladen na to, aby práce s grafy byla pohodlnější, ale aby nedocházelo ke zbytečnému kopírování dat a velké ztrátě efektivity.

Součástí práce je také rozšíření generátoru geng o podporu barevných tříd a zakotvených vrcholů. Tyto struktury umožňují generovat bohatší třídy grafů, u nichž izomorfismus musí respektovat dodatečné informace přiřazené vrcholům.

Práce je rozdělena následovně. První kapitola shrnuje teoretické pozadí generování grafů a popisuje hlavní problémy, které se při něm objevují. Druhá kapitola se věnuje nástrojům geng a plantri. Třetí kapitola popisuje návrh knihovny PlanG. Čtvrtá kapitola se zabývá implementací rozhraní a úpravami původních generátorů. Pátá kapitola obsahuje testování a vyhodnocení výsledného řešení.

Kapitola 1

Teoretický úvod

1.1 Počítačem podporovaná teorie grafů

Počítače se v teorii grafů nepoužívají pouze k numerickým výpočtům, ale také jako nástroj pro objevování a ověřování matematických tvrzení. V počítačem podporované teorii grafů můžeme například prohledávat grafy dané velikosti, vybírat z nich grafy se zadanými vlastnostmi a testovat na nich navržená tvrzení.

Typickou úlohou je hledání protipříkladů k tvrzením tvaru: každý graf s vlastností P_1 má také vlastnost P_2 . Pokud takové tvrzení neplatí, chceme najít graf, který splňuje P_1 , ale nesplňuje P_2 . Takový graf pak tvoří konkrétní protipříklad k původnímu tvrzení. [1, Section 2].

Takový postup sám o sobě nenahrazuje matematický důkaz obecného tvrzení. Je však užitečný ve fázi hledání hypotéz a protipříkladů. Pokud nemůžeme tvrzení vyvrátit na malých grafech, může nám počítač pomoci odhadnout jeho správnou formulaci a nasměrovat hledání důkazu. Pokud naopak najdeme malý protipříklad, může nám pomoci pochopit, proč tvrzení neplatí.

1.2 Generování grafů

Generováním grafů budeme rozumět postup, při kterém algoritmus vytváří grafy z předem určené třídy. V nejjednodušším případě bychom pro pevný počet vrcholů mohli projít všechny možné hrany mezi těmito vrcholy. Pro graf na n vrcholech existuje $\binom{n}{2}$ možných hran, a tedy $2^{\binom{n}{2}}$ označených neorientovaných grafů. Toto číslo roste velmi rychle, takže přímé procházení všech možností použijeme jen pro malé hodnoty n .

V mnoha případech nám navíc nezáleží na konkrétních jménech vrcholů. Dva grafy, které se liší pouze přejmenováním vrcholů, představují z hlediska mnoha vlastností grafů stejný případ. Pokud bychom generovali všechny označené grafy,

jeden graf bychom zpracovávali v mnoha různých případech. Praktické generátory se proto snaží vytvořit pouze jeden reprezentant z každé izomorfní třídy.

Generátor obvykle nepoužívá jednoduchý seznam všech možností, ale generuje graf postupně. Může například začít prázdným grafem a přidávat vrcholy nebo hrany a průběžně kontrolovat, zda částečně vytvořený graf ještě může vést ke grafu požadované třídy. Díky takovému inkrementálnímu postupu můžeme zahazovat celé větve výpočtu dříve, než je dokončíme.

1.3 Izomorfismus grafů

Dva grafy G a H nazveme izomorfní, pokud existuje bijekce mezi jejich vrcholy, která zachovává sousednost. Jinými slovy, grafy mají stejnou strukturu a liší se pouze pojmenováním vrcholů. Při generování grafů je izomorfismus zásadní, protože nechceme opakovaně zpracovávat různé popisy téhož grafu.

Pokud grafy obsahují další strukturu, například barvy vrcholů nebo vybrané zakotvené vrcholy, musí izomorfismus respektovat i tyto informace. U obarvených grafů tedy můžeme zaměňovat pouze vrcholy se stejnou barvou. Tato myšlenka je pro nás důležitá, protože v této práci rozšiřujeme generátor geng o barevné třídy a zakotvené vrcholy, které musí izomorfismus v procesu generování respektovat.

Prakticky narazíme na to, že testování izomorfismu je výpočetně náročná úloha a při generování se může objevit velmi často. Efektivní generátory proto používají specializované algoritmy a reprezentace, které umožňují rychle rozhodovat, zda jsme daný případ již nevygenerovali v jiné podobě.

1.4 Kanonické reprezentace

Jedním ze způsobů, jak pracovat s izomorfismem, je přiřadit každému grafu kanonickou reprezentaci. Chceme, aby všechny izomorfní kopie téhož grafu dostaly stejný výsledek. Pokud tedy dva grafy mají stejnou kanonickou reprezentaci, můžeme je považovat za navzájem izomorfní.

Kanonické značení se používá například při ukládání výsledků, porovnávání grafů nebo odstraňování duplicit. Generátory ho využívají uvnitř samotného algoritmu, aby nevytvářely více zástupců jedné izomorfní třídy. Jedním z takových nástrojů pro práci s kanonickým značením grafů je knihovna nauty. [2].

Kanonické značení se komplikuje ve chvíli, kdy graf nenese pouze informaci o hranách, ale také další data. Pokud mají vrcholy barvy, musí kanonické značení zachovat rozdělení vrcholů do barevných tříd. Při návrhu rozšíření pro obarvené grafy proto nestačí přidat barvy až do výstupu. Musíme je zahrnout už do testů izomorfismu.

1.5 Backtracking a prořezávání

Mnoho generátorů grafů je založeno na backtrackingu. Algoritmus postupně vytváří částečný graf a z něj zkouší všechny přípustné možnosti rozšíření. Pokud zjistíme, že daný částečný graf nemůže vést k žádnému požadovanému výsledku, vrátíme se a pokračujeme jinou větví výpočtu.

Tomuto předčasnému zahazování větví říkáme prořezávání. Prořezávání může být založeno na jednoduchých podmínkách, například na počtu hran nebo stupních vrcholů, ale také na složitějších algoritmech a heuristikách. Čím dříve dokážeme větve zahodit, tím méně práce musí generátor vykonat.

V kontextu této práce budeme rozlišovat filtrování hotových grafů a prořezávání částečně vygenerovaných grafů. Filtrování hotových grafů je jednodušší, protože pracujeme s kompletním grafem. Prořezávání během generování může být výrazně efektivnější, ale musíme počítat s tím, že pracujeme pouze s částečným grafem a některé vlastnosti ještě nemusí být rozhodnutelné.

1.6 Praktické komplikace generování grafů

První praktickou komplikací je kombinatorická exploze. I když generujeme pouze neizomorfní grafy, počet výsledných grafů může být velmi vysoký. Generátory proto musí být implementovány s důrazem na efektivitu a často používají nízkoúrovňové datové struktury, například bitové reprezentace množin hran.

Druhou komplikací je práce s interní reprezentací grafu. Výkonné generátory jsou často psané v jazyce C a jejich vnitřní struktury jsou optimalizované pro rychlost, nikoli pro pohodlné použití z aplikačního kódu. Pokud chceme nad generovanými grafy spouštět vlastní algoritmy, často musíme psát pluginy nebo pracovat přímo s detaily implementace generátoru.

Třetí komplikací je propojení generování s dalšími algoritmy. V experimentální práci běžně chceme každý vygenerovaný graf ihned testovat, obarvovat, porovnávat nebo převádět do jiného formátu. Pokud kvůli tomu musíme graf kopírovat do jiné datové struktury, můžeme ztratit podstatnou část výkonu původního generátoru.

Tyto problémy motivují návrh rozhraní, které zachová výkon specializovaných generátorů, ale současně poskytne pohodlnější způsob konfigurace, filtrování a zpracování grafů.

Kapitola 2

Generátory grafů geng a plantri

V předchozí kapitole jsme popsali obecný problém generování grafů bez izomorfních kopií. V této kapitole se podíváme na dva konkrétní generátory, na kterých je tato práce postavena: geng z balíku nauty a plantri. Oba nástroje řeší podobný typ úlohy, ale pracují s jinými třídami grafů a používají jiné interní reprezentace. Pro návrh jednotného rozhraní proto potřebujeme porozumět nejen tomu, jaké grafy generují, ale také tomu, jak se do jejich běhu dá zasáhnout z vlastního programu.

2.1 Generátor geng

Generátor geng je součástí balíku nauty. Balík nauty je známý především algoritmy pro práci s automorfismy grafů a s kanonickým značením [2]. Program geng využívá tyto algoritmy pro generování neizomorfních neorientovaných grafů na zadaném počtu vrcholů. Uživatel může omezit například počet hran, souvislost, minimální a maximální stupeň vrcholů nebo zakázat vybrané malé podgrafy (například trojúhelníky či čtyřcykly).

Takový graf můžeme popsat množinou vrcholů a množinou hran. Při izomorfismu nás tedy zajímá pouze zachování sousednosti. Reprezentace grafu v nauty je proto založena na bitových množinách sousedů, které jsou velmi kompaktní a dobře se hodí pro rychlé množinové operace.

Z pohledu uživatele na příkazové řádce je práce s geng poměrně přímočará: zadáme počet vrcholů a přepínače popisující požadovanou třídu grafů. Uvnitř však geng generuje grafy inkrementálně: z grafu na n vrcholech vytváří graf na $n + 1$ vrcholech tak, že přidá nový poslední vrchol a zvolí jeho sousedy mezi již existujícími vrcholy. Aby přitom nevznikaly izomorfní kopie téhož grafu, nepřijme žádné možné rozšíření. Nově vzniklý graf musí projít kanonickým testem, který mimo jiné rozhoduje, zda právě poslední přidaný vrchol může být

považován za kanonicky zvolený. Při tomto rozhodování se používají i stupně vrcholů. Zjednodušeně řečeno, generátor nepracuje se všemi pořadími vzniku vrcholů stejně, ale využívá vlastnosti posledního vrcholu k tomu, aby omezil počet zkoušených možností.

Při rozšíření generátoru o barvy je důležité, že barvy musí ovlivňovat také testy izomorfismu a kanonické rozhodování během generování. Konkrétní úpravy potřebné pro tuto podporu popíšeme v kapitole o implementaci.

2.2 Generátor plantri

Generátor plantri je specializovaný nástroj pro generování rovinných grafů. Umí generovat například rovinné triangulace, kvadrangulace, jednoduché rovinné grafy, triangulace disku. Stejně jako geng negeneruje všechny možné označené grafy, ale jeden graf pro každou izomorfní třídu.

Generování v plantri je založeno na postupném zvětšování rovinného grafu pomocí lokálních operací, které se nazývají expanze. Opačné operace, které graf zmenšují, se nazývají redukce [3, Section 1.2]. Jednoduše řečeno, expanze je krok, kterým graf zvětšíme, zatímco redukce je odpovídající krok, kterým jej zmenšíme.

Například při generování triangulací můžeme začít od malého počátečního grafu a postupně jej zvětšovat operacemi, které přidávají vrchol nebo upravují lokální okolí některé stěny. Každé takové rozšíření zachovává rovinné vložení a strukturu triangulace. Protože stejný výsledný graf může vzniknout více různými posloupnostmi expanzí, musí generátor průběžně odmítat nekanonické větve výpočtu.

Tento problém řeší metoda kanonické konstrukční cesty [3, Section 1.2]. Pro nově vzniklý graf se určí kanonicky zvolená redukce, tedy opačný krok, kterým by se graf měl zmenšit. Rozšíření se přijme pouze tehdy, pokud právě provedená expanze odpovídá této kanonické redukci. Díky tomu generátor vytvoří pouze jeden reprezentant z každé izomorfní třídy.

plantri generuje rovinné grafy spolu s jejich vložení. U takového grafu nestačí vědět, které dvojice vrcholů jsou spojeny hranou. Potřebujeme také znát cyklické pořadí hran kolem každého vrcholu, protože právě toto pořadí určuje stěny rovinného vložení. plantri proto používá reprezentaci založenou na orientovaných hranách. Každá neorientovaná hrana je reprezentována dvojicí opačně orientovaných hran. Každá taková orientovaná hrana obsahuje odkazy na následující a předchozí hranu v cyklickém pořadí kolem vrcholu.

Taková half-edge reprezentace umožňuje přirozeně procházet okolí vrcholu i stěny rovinného vložení.

2.3 Možnosti rozšíření pomocí callbacků

Oba generátory umožňují zasáhnout do generování vlastním kódem, ale dělají to způsobem typickým pro C programy. geng nabízí například mechanismy OUTPROC, PRUNE a PREPRUNE. OUTPROC umožňuje nahradit standardní výpis grafu vlastní funkcí. PRUNE a PREPRUNE umožňují odmítat průběžně vytvářené grafy a tím prořezávat prohledávací strom.

plantri používá podobnou myšlenku přes pluginy a makra jako FILTER a PRE_FILTER. Uživatel může doplnit vlastní testy, které se spouštějí při generování, a rozhodovat, zda má být graf vypsán nebo zda má být větev výpočtu zahozena. Díky tomu můžeme ušetřit více času než při filtrování hotového výstupu, protože nevhodné části prohledávacího prostoru zahodíme dříve.

Tyto mechanismy jsou výkonné, ale jejich přímé použití z C++ vyžaduje znalost přesných signatur callbacků, způsobu sestavení generátoru s příslušnými makry a často také interní reprezentace aktuálního grafu.

2.4 Omezení původních rozhraní

Původní rozhraní generátorů geng a plantri je velmi vhodné pro použití z příkazové řádky. Pokud chceme pouze vygenerovat grafy dané třídy a uložit je do souboru, jsou oba nástroje rychlé a praktické. Pro použití jako součást většího C++ programu však narážíme na několik omezení.

Prvním omezením je rozdílný způsob konfigurace. Každý generátor má vlastní sadu prepínačů, vlastní výstupní formáty a vlastní pojmenování podobných voleb. Druhým omezením je nízkoúrovňové C rozhraní. Pokud chceme generátor volat přímo z programu, musíme připravovat argumenty ve tvaru podobném příkazové řádce a pracovat s globálním stavem původního generátoru.

Třetím omezením je reprezentace grafů v callbackách. U geng pracujeme s bitovou reprezentací nauty, u plantri s poli stupňů a ukazateli na orientované hrany. Obě reprezentace jsou efektivní, ale uživatel musí znát jejich detaily, pokud chce nad grafem spustit vlastní algoritmus. Převod každého grafu do jiné datové struktury by rozhraní zjednodušil, ale při velkém počtu generovaných grafů by mohl znamenat výraznou ztrátu výkonu.

Při návrhu rozhraní proto chceme tato omezení skrýt, aniž bychom ztratili výhody původních generátorů.

Kapitola 3

Návrh rozhraní

V předchozí kapitole jsme popsali způsoby rozšíření a omezení původních generátorů `geng` a `plantri`. V této kapitole popíšeme veřejné C++ rozhraní knihovny `PlanG`.

3.1 Cíle návrhu

Původní generátory použijeme jako backendy a nad nimi nabídneme společné C++ rozhraní. Uživatel si zvolí požadovaný backend, nastaví parametry generování a připojí vlastní funkce pro filtrování nebo zpracování grafů.

Součástí návrhu je také zpracování grafů bez automatického převodu do nové datové struktury. Vygenerovaný graf proto zpřístupníme pomocí view nad daty původního generátoru.

3.2 Jednotné rozhraní pro více generátorů

Základním vstupním bodem knihovny je šablona `Generator<Backend>`. Parametr `Backend` určuje, který původní generátor se použije. Pro obecné grafy tedy pracujeme s typem `Generator<geng::Backend>`, zatímco pro rovinné grafy s typem `Generator<plantri::Backend>`.

Toto uspořádání využívá princip *policy-based design*. Společná část rozhraní je implementována šablonou `Generator`, která poskytuje například metodu `run()` a metody pro registraci callbacků. Typ `Backend` doplňuje funkce a nastavení specifická pro zvolený generátor.

Uživatel proto pracuje s oběma generátory stejným způsobem; liší se pouze zvolený backend a nastavení dostupná pro daný generátor. Toto rozdělení také umožňuje případné doplnění dalšího generátoru vytvořením odpovídajícího backendu.

3.3 Callbacky pro filtrování a zpracování grafů

Při generování grafů často nechceme pouze vypsat všechny grafy do souboru. Chceme nad každým grafem spustit vlastní test, zahodit grafy, které nás nezajímají, nebo průběžně ukládat jen vybrané výsledky. Proto rozhraní obsahuje tři základní typy callbacků.

Callback nastavený pomocí `setOutproc` slouží ke zpracování hotového grafu. Můžeme ho použít například pro vlastní výpis, sběr statistik nebo spuštění grafového algoritmu nad vygenerovaným grafem. Callback `setPrune` umožňuje odmítnout hotový graf. Callback `setPreprune` je určen pro prořezávání rozpracovaných větví během konstrukce.

Tato trojice odpovídá možnostem původních generátorů, ale uživatel s nimi pracuje přes C++ funkční objekty. Nemusí tedy psát samostatný C plugin ani kompilovat generátor s vlastními makry. Z pohledu návrhu je důležité, že callback nedostává kopii grafu, ale view na právě generovaný graf.

3.4 View na graf bez kopírování dat

Třída `GraphView` představuje nevlastnící view na graf uložený uvnitř původního generátoru. U backendu `geng` ukazuje na reprezentaci grafu v `nauty`, U backendu `plantri` ukazuje na pole stupňů a strukturu orientovaných hran. V obou případech je cílem zpřístupnit graf přes srozumitelnější operace, aniž bychom museli vytvářet jeho úplnou kopii.

Takové view je určeno hlavně pro použití uvnitř callbacku. Uživatel může procházet vrcholy, hrany nebo sousedy vrcholu, ale nemá vlastnictví nad daty generátoru. Proto `GraphView` nemá být dlouhodobě ukládán mimo callback, protože po pokračování generování se interní data mohou změnit.

Výhodou tohoto návrhu je nízká režie. Uživatel má k dispozici data zpřístupněná objektem `GraphView` a sadu `free` funkcí pro práci s grafem.

3.5 Integrace s Boost Graph Library

Boost Graph Library je založena na grafových konceptech, které popisují, jaké typy a funkce musí graf poskytovat, aby nad ním šly spouštět obecné grafové algoritmy [4]. Důležité je, že graf nemusí být konkrétní třídou z Boostu. Stačí, aby poskytoval požadované rozhraní.

To dobře odpovídá našemu cíli. Objekt `GraphView` navrhujeme tak, aby jej bylo možné použít s vybranými algoritmy `Boost.Graph` přímo uvnitř callbacku. Uživatel tak může nad vygenerovaným grafem spouštět standardní grafové algoritmy v

C++, aniž by graf převáděl do nové datové struktury nebo znal interní reprezentaci použitého generátoru.

Konkrétní implementaci tohoto rozhraní pro oba backendy popíšeme v kapitole 4.

3.6 Podpora bohatších struktur

Backend geng umožňuje generovat grafy s barevnými třídami vrcholů. Uživatel může nastavit počet barev, přesné velikosti barevných tříd nebo jejich dolní a horní meze. Rozhraní nabízí také samostatnou volbu pro zakotvené vrcholy.

Callbacks mohou při zpracování grafu číst přiřazené barvy prostřednictvím `GraphView`. Způsob předání barev do původního generátoru a jejich zahrnutí do generování popíšeme v kapitole o implementaci.

Kapitola 4

Implementace

V této kapitole popíšeme implementaci knihovny PlanG. V předchozí kapitole jsme navrhli rozhraní, které má sjednotit práci s generátory geng a plantri. Nyní se podíváme na to, jak je toto rozhraní napojeno na původní C programy a jakým způsobem zpřístupňujeme grafy uživatelským callbackům.

C++ část knihovny převádí konfiguraci uloženou v objektu na argumenty původního programu. Callbacky a konfiguraci obarvení, které nejsou součástí argumentů příkazové řádky, předává před spuštěním generátoru do C mezivrstvy. Aktuální graf následně zpřístupňuje pomocí view nad interními daty generátoru.

4.1 Architektura knihovny

Implementace je rozdělena do několika vrstev. Nejnižší zůstávají původní zdrojové kódy generátorů. Tyto části obsahují samotné algoritmy pro generování grafů a knihovna je nepřepisuje, s výjimkou úprav generátoru geng, které doplňují podporu barevných tříd a zakotvených vrcholů.

Nad nimi je tenká C mezivrstva. Tato vrstva zpřístupňuje vybrané funkce a globální proměnné původních generátorů a umožňuje zaregistrovat callbacky z C++ části programu.

Nad C mezivrstvou jsou implementovány C++ backendy. Každý backend uchovává konfiguraci generování, kontroluje její základní správnost a umí ji převést na argumenty původního generátoru. Backend také před spuštěním registruje callbacky a předává informace o barvách vrcholů.

Nejvyšší vrstvu tvoří šablona `Generator<Backend>`, která uživateli poskytuje společné rozhraní nezávislé na konkrétním generátoru.

4.2 Vrstva Generator a backendy

Šablona `Generator<Backend>` dědí z konkrétního backendu. Díky tomu jsou metody backendu dostupné přímo na objektu generátoru, ale společná vrstva může zároveň definovat operace, které mají všechny backendy stejné. Patří sem zejména metoda `run` a metody pro registraci callbacků `setPrune`, `setPreprune` a `setOutproc`.

Při spuštění generování metoda `run` zavolá backendovou metodu `prepare_args`. Metoda `prepare_args` vytvoří seznam řetězců které odpovídají argumentům příkazové řádky původního programu. Poté se zavolá `apply_runtime_state`, která nastaví callbacky a případně nastaví konfiguraci obarvení. Nakonec se seznam řetězců převede na pole ukazatelů typu `char*` a předá se původní vstupní funkci generátoru.

Konkrétní backend tedy plní dvě role. Navenek nabízí typovější C++ metody pro nastavení parametrů (počet vrcholů, omezení stupňů nebo výstupní formát). Uvnitř ví, jak tyto volby vyjádřit pomocí přepínačů původního programu. Díky tomu uživatel nemusí skládat argumenty příkazové řádky ručně, a zároveň zůstává zachován původní způsob spuštění generátorů.

4.3 Propojení s geng

Backend pro `geng` uchovává parametry odpovídající přepínačům původního programu. Například počet vrcholů, rozsah počtu hran, požadavky na souvislost, omezení minimálního a maximálního stupně, volba výstupního formátu nebo rozdělení výpočtu na části. Metoda `prepare_args` tyto hodnoty převede na argumenty, které by uživatel jinak zadal na příkazové řádce.

Samotné spuštění probíhá přes přejmenovanou vstupní funkci původního programu. Kód `geng` je při sestavení upraven tak, aby jeho hlavní funkce nebyla `main`, ale funkce volatelná z knihovny. Zároveň je generátor sestaven s makry `OUTPROC`, `PRUNE` a `PREPRUNE`, která směřují do C mezivrstvy. Díky těmto makrům se výpis nalezených grafů a volání prořezávacích testů přesměrují z původního programu do funkcí knihovny.

Volání z `geng` projdou přes C mezivrstvu do trampoline funkcí v C++ části. C++ část uchovává uživatelské callbacky a při jejich volání vytvoří `geng::GraphView`. Ten obsahuje ukazatel na graf v reprezentaci nauty, počet vrcholů aktuálního grafu a cílový počet vrcholů.

Důležitý rozdíl oproti původnímu `PRUNE` mechanismu je v tom, jakou sémantiku nabízíme uživateli. V původním `geng` se `PRUNE` volá i pro mezilehlé grafy během konstrukce. V rozhraní `PlanG` chceme, aby `setPrune` filtroval hotové grafy, zatímco `setPreprune` sloužil k prořezávání rozpracovaných větví. C++

napojení proto u callbacku `setPrune` nejprve ověří, zda aktuální počet vrcholů odpovídá cílovému počtu vrcholů. Pokud podmínka není splněna, uživatelský callback se nezavolá.

4.4 Propojení s plantri

Backend pro `plantri` pracuje podobným způsobem, ale napojuje se na jiné vnitřní mechanismy původního generátoru. Konfigurace obsahuje například volbu třídy generovaných grafů, omezení minimálního stupně a souvislosti, volbu výstupního formátu, generování duálních grafů nebo rozdělení výpočtu. Metoda `prepare_args` z těchto voleb sestaví argumenty pro původní program `plantri`.

Callbacks v `plantri` jsou navázány na makra `FILTER` a `PRE_FILTER`. C mezivrstva je přesměruje do trampoline funkcí v C++ části, které pak volají callbacky nastavené backendem. Callback `setPrune` odpovídá mechanismu `FILTER` a pracuje s kandidátem na výstupní graf. Přes stejnou trampoline funkci obsluhujeme také `setOutproc`, který se zavolá při zpracování nalezeného grafu. Callback `setPreprune` odpovídá mechanismu `PRE_FILTER` a může být volán nad rozpracovaným grafem během rekurzivní konstrukce.

Protože `plantri` drží aktuální graf v globálních proměnných, C mezivrstva zpřístupňuje funkce pro čtení těchto hodnot. C++ vrstva z nich sestaví view pro backend `plantri`. Do něj se předávají ukazatele na hrany, pole stupňů vrcholů, počet vrcholů, počet orientovaných hran a cílový počet vrcholů.

4.5 Implementace GraphView

Pro každý backend je implementován samostatný typ `GraphView`, protože oba generátory ukládají graf odlišným způsobem. Typ `geng::GraphView` uchovává ukazatel na graf v bitové reprezentaci nauty a počet jeho vrcholů. Při průchodu grafem iterátory procházejí sousedy a hrany přímo v této reprezentaci.

Typ `plantri::GraphView` pracuje s poli stupňů a s orientovanými hranami původního rovinného vložení. Při procházení hran využívá odkazy mezi orientovanými hranami. Počítá také s hodnotou `missing_vertex`, protože některé vnitřní reprezentace `plantri` mohou přeskočit jeden index vrcholu.

4.5.1 Integrace s Boost Graph Library

Aby bylo možné nad objektem `GraphView` spouštět algoritmy z knihovny `Boost Graph Library`, implementujeme pro oba backendy rozhraní požadované grafovými koncepty této knihovny. Oba typy `GraphView` poskytují typy používané

šablonou `boost::graph_traits`¹, například typ vrcholu, typ hrany, iterátory a podporované způsoby průchodu grafem.

Pro oba backendy dále implementujeme free funkce požadované grafovými koncepty `Boost.Graph` [4], například `vertices`, `edges`, `num_vertices`, `out_edges`, `adjacent_vertices`, `source` a `target`. Tyto operace pracují přímo s daty zpřístupněnými příslušným view a nevytvářejí kopii grafu.

Součástí integrace je také mapa indexů vrcholů. U backendu `geng` používáme identické mapování, protože vrcholy jsou číslovány souvisle od nuly. U backendu `plantri` vlastní mapa indexů přeskočí případný chybějící vrchol (`missing_vertex`). Algoritmy z `Boost Graph Library` tak mohou pracovat s oběma backendy stejným způsobem.

4.6 UML diagram tříd

4.7 Obarvené a zakotvené vrcholy v geng

4.8 Callbacky a přechod mezi C a C++

4.9 Výstupní formáty

¹https://www.boost.org/doc/libs/latest/libs/graph/doc/graph_traits.html

Kapitola 5

Testování a vyhodnocení

Conclusion

In the conclusion, you should summarize what was achieved by the thesis. In a few paragraphs, try to answer the following:

- Was the problem stated in the introduction solved? (Ideally include a list of successfully achieved goals.)
- What is the quality of the result? Is the problem solved for good and the mankind does not need to ever think about it again, or just partially improved upon? (Is the incompleteness caused by overwhelming problem complexity that would be out of thesis scope, or any theoretical reasons, such as computational hardness?)
- Does the result have any practical applications that improve upon something realistic?
- Is there any good future development or research direction that could further improve the results of this thesis? (This is often summarized in a separate subsection called 'Future work'.)

This is quite common.

Seznam použité literatury

- [1] Jorik Jooken. *Computer-assisted graph theory: a survey*. 2025. URL: <https://arxiv.org/pdf/2508.20825>.
- [2] Brendan D. McKay a Adolfo Piperno. *nauty and Traces*. URL: <https://pallini.di.uniroma1.it/>.
- [3] McKay Brinkmann Gunnar a Brendan D. “Fast generation of planar graphs”. In: (). URL: <https://users.cecs.anu.edu.au/~bdm/papers/plantri-full.pdf>.
- [4] Boost C++ Libraries. *Boost Graph Library: Graph Concepts*. URL: https://www.boost.org/doc/libs/latest/libs/graph/doc/graph_concepts.html.

Příloha A

Using CoolThesisSoftware

Use this appendix to tell the readers (specifically the reviewer) how to use your software. A very reduced example follows; expand as necessary. Description of the program usage (e.g., how to process some example data) should be included as well.

To compile and run the software, you need dependencies XXX and YYY and a C compiler. On Debian-based Linux systems (such as Ubuntu), you may install these dependencies with APT:

```
apt-get install \  
  libsuperdependency-dev \  
  libanotherdependency-dev \  
  build-essential
```

To unpack and compile the software, proceed as follows:

```
unzip coolsoft.zip  
cd coolsoft  
./configure  
make
```

The program can be used as a C++ library, the simplest use is demonstrated in výpis 1. A demonstration program that processes demonstration data is available in directory demo/, you can run the program on a demonstration dataset as follows:

```
cd demo/  
./bin/cool_process_data data/demo1
```

After the program starts, control the data avenger with standard WSAD controls.

Výpis kódu 1 Example program.

```
#include <CoolSoft.h>
#include <iostream>

int main() {
    int i;
    if(i = cool::ProcessAllData()) // returns 0 on error
        std::cout << i << std::endl;
    else
        std::cerr << "error!" << std::endl;
    return 0;
}
```
